

High Performance Programming

Lecture 8:
“GPU programming”
Lecturer: Stefan



AALBORG
UNIVERSITET

0. GPU setup

Contents

- 1.Introduction
- 2.The many flavors of GPU programming
- 3.PyTorch and Numba
- 4.PyOpenCL
- 5.Applications and examples
- 6.Summary and exercises

1.Introduction

Graphics Processing Units (GPUs)

Made to manipulate pixels to display pictures on a screen

Pixels are not known in advance, so GPUs must respond fast

If we have a 1920x1080 pixel RGB display updated at 60 fps

We need to produce

$$(1920 \times 1080) \times 60 \times 3 = 373.248 \text{ million pixels per second}$$

GPUs are made of simple processing elements with a reduced set of instructions

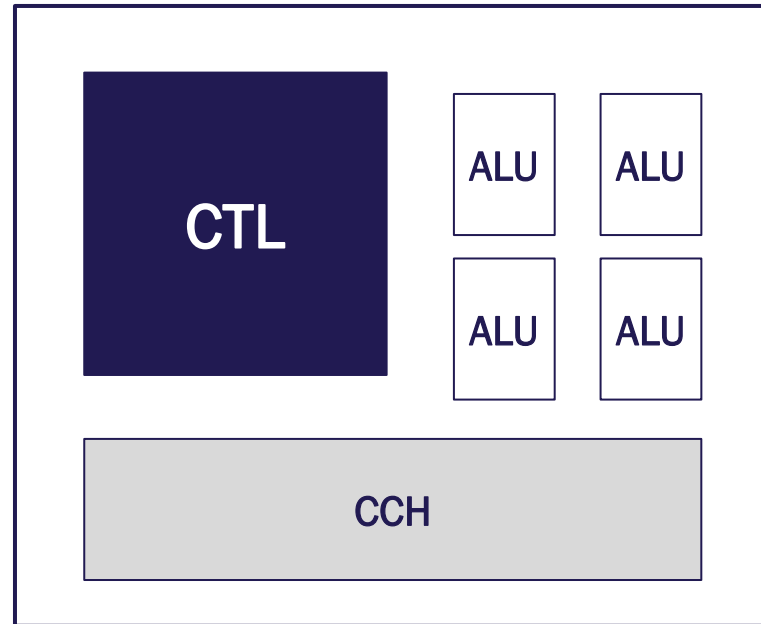
It all started with games... and by the 2000s scientists started using GPUs

Machine learning and artificial intelligence are to blame



Architectures: CPU vs. GPU

CPU

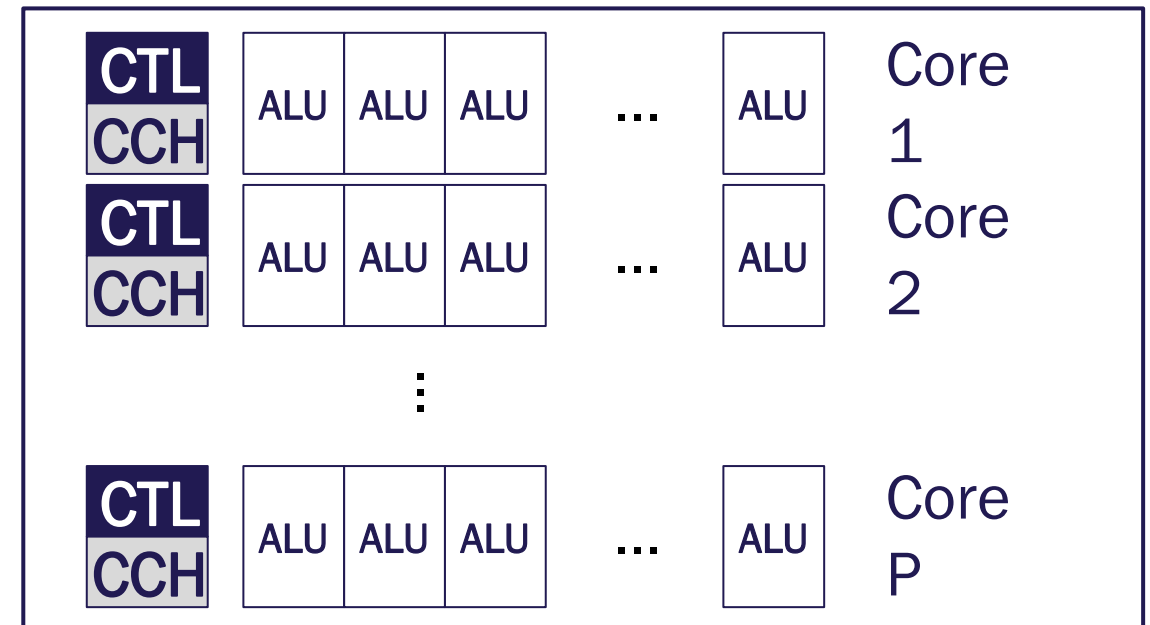


Large area to control (CTL) and cache (CCH)

Few cores

Few arithmetic and logic units (ALUs)

GPU



Low control capabilities

Large number of cores

Massive number of ALUs

Purpose of the GPU

Perform a large number of similar operations at once

Single Instruction Multiple Data (SIMD)

Specifically, Single Instruction Multiple Threads (SIMT)

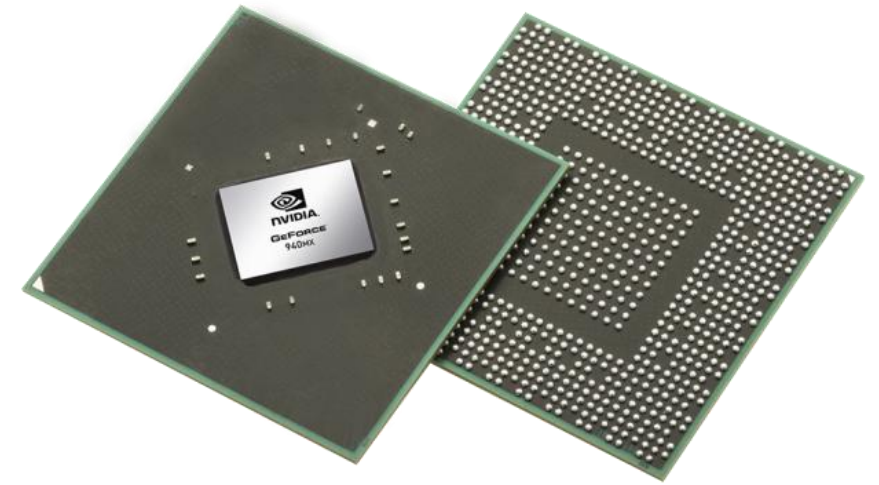
Cores work synchronously but distinct operations can be done in each core

Great for problems with a large number of common operations

Video and image processing, simulations, and machine (deep) learning

Not good if

- Large amounts of control operations are needed
- Communication from CPU to GPU becomes the bottleneck
- Complex memory access patterns are needed: branching
- **Vectorized operations are available**



TensorFlow

 PyTorch

 Keras

The many flavors of GPU programming

NVIDIA Compute Unified Device Architecture (CUDA)

Supported languages: C, C++, Fortran, Python and MATLAB

CUDA Toolkit (not a Python package)

Libraries, compiler, development tools

AMD ROCm™ open software platform: [webpage](#)

OpenCL is a framework for writing programs that execute across heterogeneous platforms

Kernel: defines the instruction and the element that must perform the operation



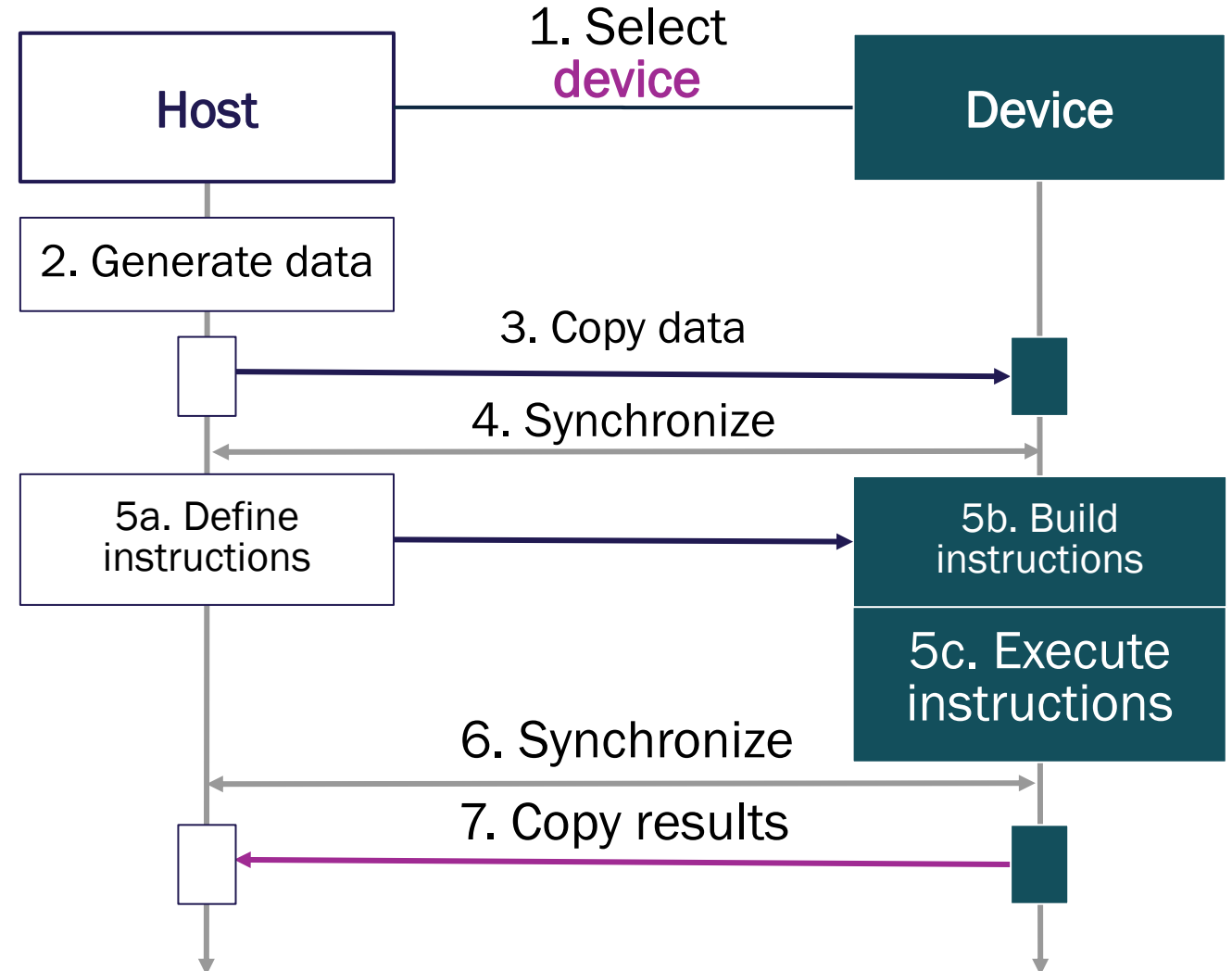
General framework for GPU programming

Host:

- Generates data
- Distributes the tasks.
- High control capabilities.

Device:

- Executes the instructions



OpenCL platform model

OpenCL is a framework for writing programs that execute across heterogeneous platforms

Host connected to **Compute Devices (CDs)**:

Each CD is a GPU/CPU

Each CD has one or more Compute Units (CUs):

Each CU is a core with one or more Processing Elements (PEs)

Each PE is an ALU

Big idea: replace loops with functions (a **kernel**)

Executing these at each point in a problem domain

OpenCL can use either GPUs or CPUs

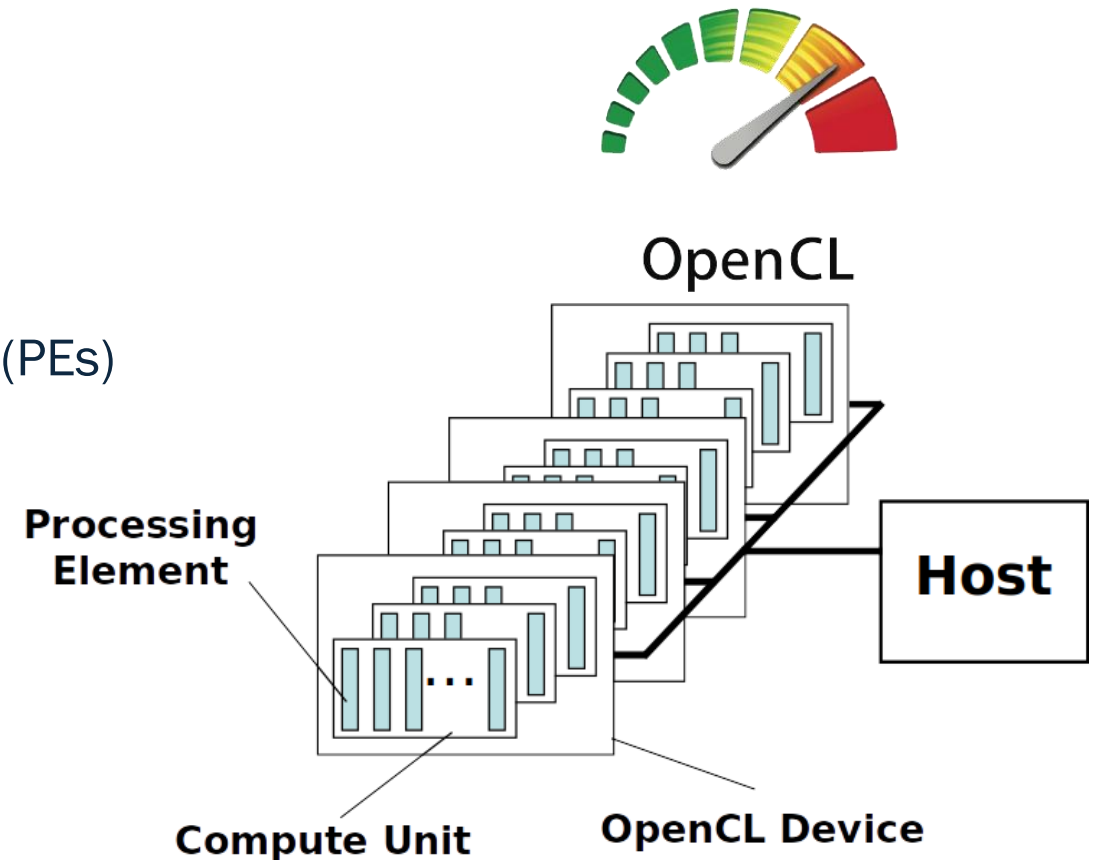


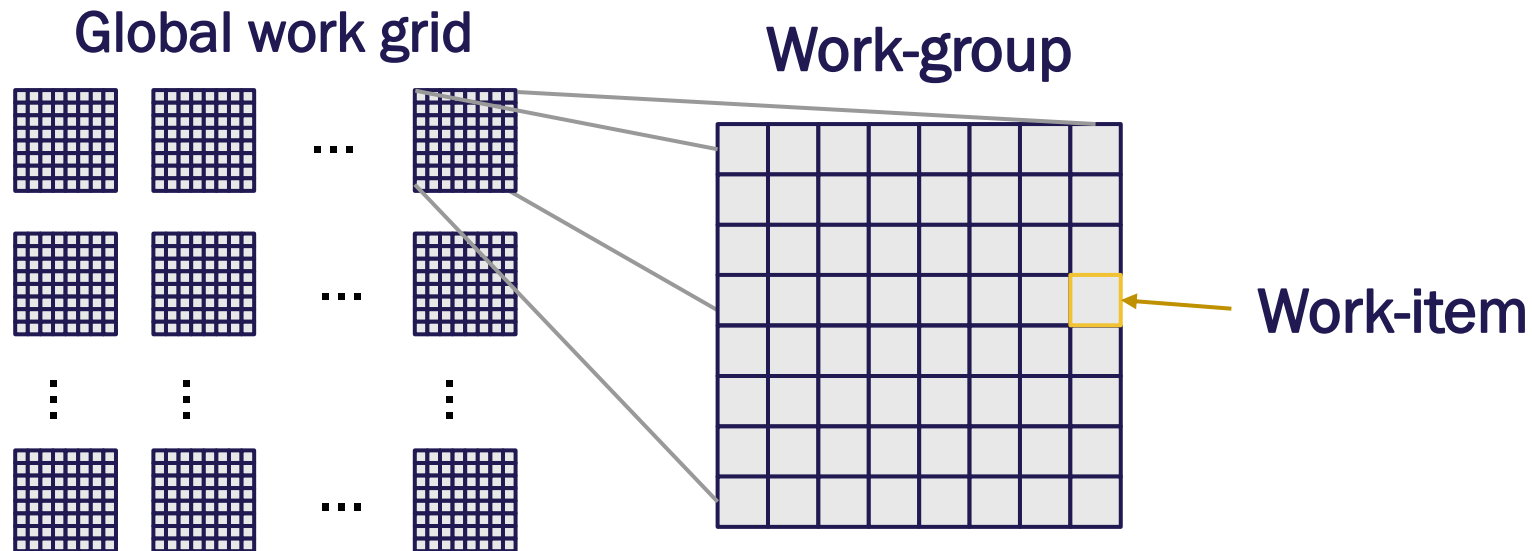
Image credit: Hands On OpenCL slides by Simon McIntosh-Smith and Tom Deakin

Work-groups and work-items

Global dimensions (whole problem space)

Work-group: executes together; is a core of the GPU

Work-item: inside work-group, can synchronize within work-group with barriers



Kernel

The problem to solve should have some dimensionality

A kernel defines the instructions and how these are distributed across the platform

When we execute a **kernel** we

- Specify up to 3 dimensions
- **Global size:** Total problem size in each dimension
- Associate each point in the iteration space with a **work-item**

Kernel

Hadamard product in C++ using traditional loops

```
void hadamard(const int n, const float *a, const float *b, float *c){  
    int i;  
    for (i = 0; i < n; i++)  
        c[i] = a[i] * b[i];  
}
```

Kernel in OpenCL

```
__kernel void mul(__global const float *a, __global const float *b,  
                  __global float *c){  
    int id = get_global_id(0);  
    c[id] = a[id] * b[id];  
}
```

Many kernel instances (work-items) execute in parallel.

OpenCL memory model

Private memory: per work-item (ALU)

Local memory: shared within a work-group (core)

Global & Constant memory: Visible to all work-groups

Host memory: On the CPU

Memory management is **explicit**: you move the data

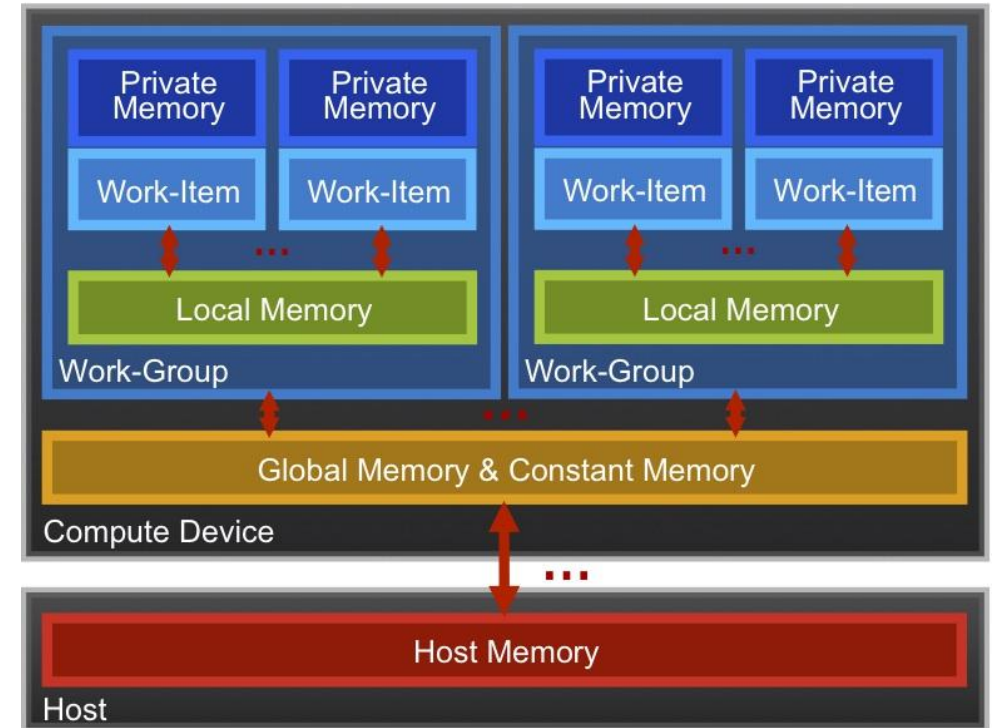


Image credit: Hands On OpenCL slides by Simon McIntosh-Smith and Tom Deakin

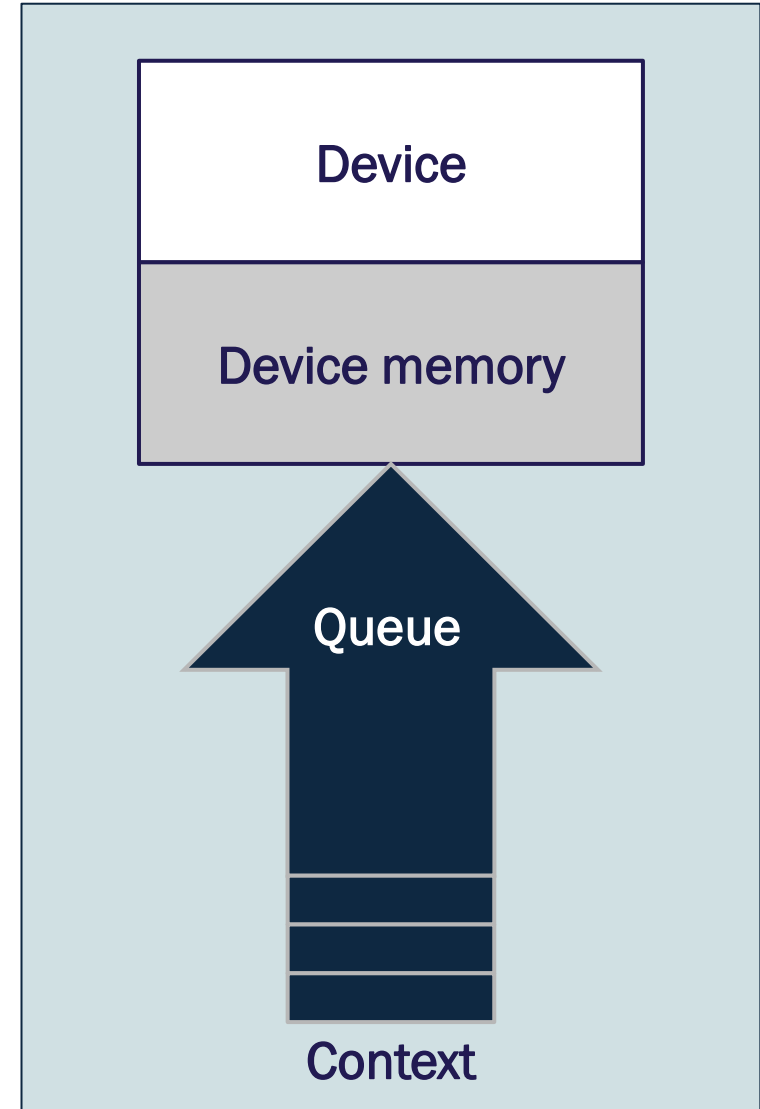
Context and command queues

Context:

- Environment where kernels execute.
- Synchronization and memory management

Includes:

- One or more devices
- Device memory
- One or more command queues



Commands are submitted through a **queue**, which points to a single device within the context

3. PyTorch and Numba

GPU programming with PyTorch



0. Import module

```
import torch
```

1. Select device

```
if torch.cuda.is_available():  
    device = torch.device("cuda")  
else:  
    device = torch.device("cpu")
```

2. Generate data at host

```
x = torch.randn(1,1000)  
A = torch.randn(1000,1000)
```

3. Copy data to device

```
x_gpu = x.to(device)  
A_gpu = A.to(device)
```

4. Synchronize

```
torch.cuda.synchronize()
```

5. Execute instructions at device

```
y_gpu = torch.matmul(x_gpu, A_gpu)
```

6. Synchronize

7. Copy data back to host

```
y = y_gpu.to("cpu")
```

Select device and device properties

Select device

```
if torch.cuda.is_available():  
    device = torch.device("cuda")  
else:  
    device = torch.device("cpu")
```

Device properties:

```
torch.cuda.get_device_properties(torch.cuda.current_device())
```

Returns:

- Name: Device name
- CUDA capabilities: A number representing the version. Mapped to features [here](#)
- Total amount of memory
- No. of multiprocessors: These can execute in parallel

GPU programming with Numba

Vectorized functions can directly be run in the GPU with CUDA capabilities

from numba import vectorize, **cuda**

Just use the decorator

```
@vectorize([ 'float32(float32,float32)' ], target = 'cuda')
```

```
def function(x,y):
```

```
    ...
```

```
    return z
```



Note: the built-in CUDA target for Numba is recently deprecated but still supported.
The CUDA target is now being maintained in the separate numba-cuda package.

<https://numba.readthedocs.io/en/stable/cuda/overview.html>

4. PyOpenCL

PyOpenCL and GPU drivers

Test installation by running

```
import pyopencl as cl
print(cl.get_platforms())
```

My output

```
[<pyopencl.Platform 'Oclgrind' at 0x21e8860>,
<pyopencl.Platform 'NVIDIA CUDA' at 0x223bb40>,
<pyopencl.Platform 'Portable Computing Language' at 0x7f9025e29320>]
```

Oclgrind:

Platform to enable the creation of developer tools for analysis and debugging of OpenCL programs. Simulates how OpenCL kernels execute. More info [here](#)

Choose platform

Initializing PyOpenCL

We need to create a **context** that specifies:

- Our environment
- The devices we are using
- Which platform we are executing on

```
import numpy as np
import pyopencl as cl
context = cl.create_some_context()
```

Based on this context, we create the command queue

```
queue = cl.CommandQueue(context)
```

How does the **context** look like?

```
<pyopencl.Context at 0x124b760 on
<pyopencl.Device 'NVIDIA GeForce
940MX' on 'NVIDIA CUDA' at
0x124ba70>>
```

4 steps to execute a kernel

1. Allocate device memory

Using a Buffer class

2. Copy input to device

Can be done simultaneously to 1

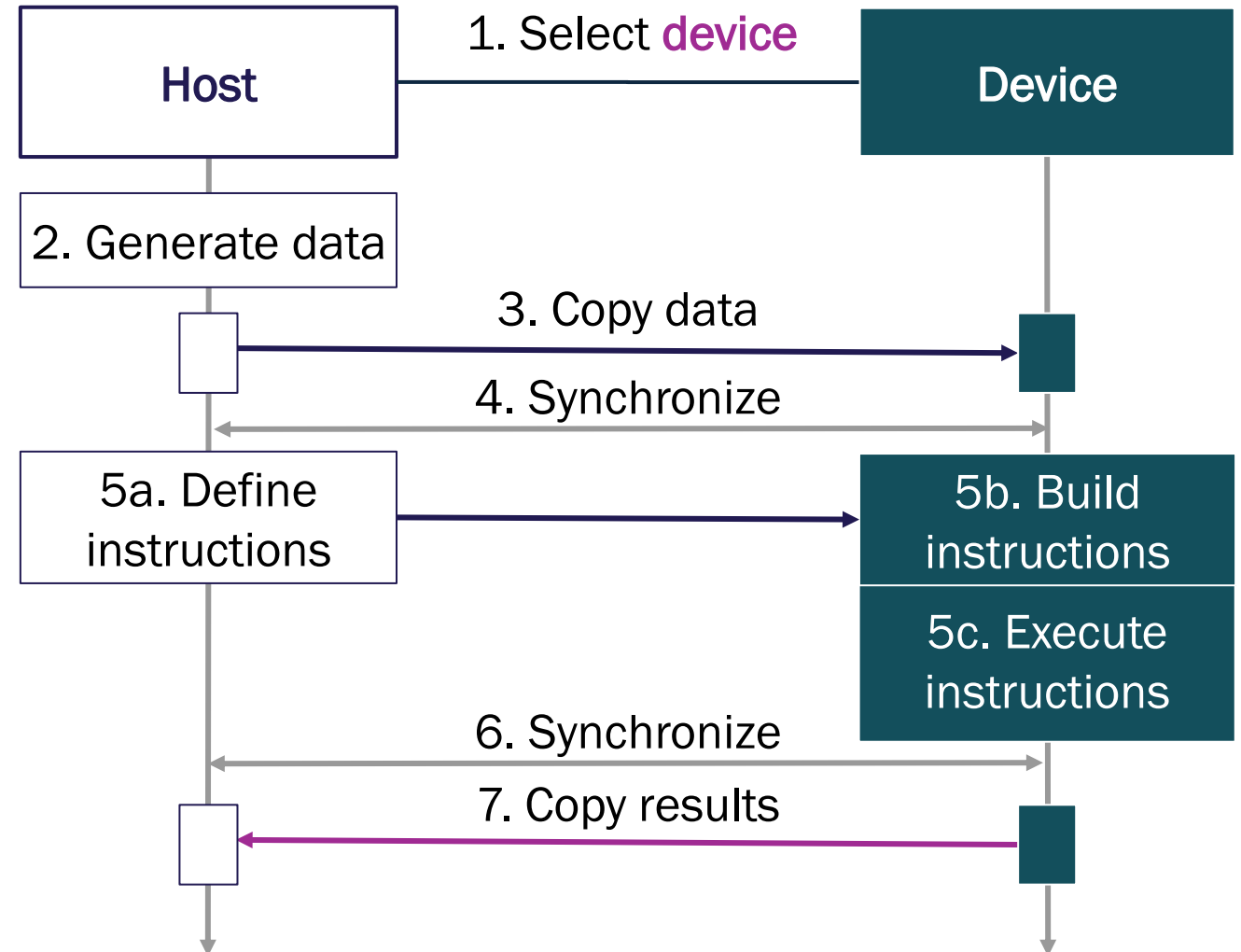
Steps 3 and 4 of our framework

3. Build and launch the kernel

Step 5 of our framework

4. Copy the result back to host

Steps 6 and 7 of our framework



1. Allocate device memory and 2. Copy input to device

This is done using the Buffer class

```
input_host = np.random.rand(DIM).astype(np.float32)
result_host = np.zeros(DIM).astype(np.float32)

mf = cl.mem_flags

input_g = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR,
                    hostbuf=input_host)

result_g = cl.Buffer(context, mf.WRITE_ONLY, result_host.nbytes)
```

By giving initial input to the host, step 2 is automatic (steps 3 and 4 of general framework)

3. Build and launch the kernel

Build (load the kernel from a file)

```
kernel_source = open("kernel.cl").read()

prg = cl.Program(context, kernel_source).build()
```

Launch

```
prg.sum(queue, input_host.shape, None, input_g, result_g)
```

Parameters

■ queue

- Global size
- Local size

*args: arguments to kernel

3. Build and launch the kernel

We need to get the work-item global ID (unique)

```
int i = get_global_id(uint dimindx); % for dimension dimindx
```

`dimindx` must be within the number of dimensions in use

These are given by `get_work_dim()`

The operation is performed in the element that corresponds to the global ID

```
output[i] = i;
```

Link to the work item functions

https://registry.khronos.org/OpenCL/sdk/3.0/docs/man/html/get_global_id.html

4. Copy the result back to the host

We must move the result from `result_g` to `result_host`

```
cl.enqueue_copy(queue, result_host, result_g)
```

Now we are back to Python

Extra: if you want to do something similar with CUDA

Just as with PyOpenCL, you can also try to use CUDA with Python.

- A good starting point for raw CUDA C++:
[An Even Easier Introduction to CUDA | NVIDIA Technical Blog](#)
- Nvidia's cuda-python library*:
 - [GitHub - NVIDIA/cuda-python: CUDA Python: Performance meets Productivity](#)
 - [CUDA Python 12.8.0 documentation](#)
- Another alternative is CuPy from Preferred Networks:
[CuPy: NumPy & SciPy for GPU](#)

Generally, you will find that this works similarly to PyOpenCL.

*You may find mentions of "pyculib" on the internet, but this is no longer maintained.

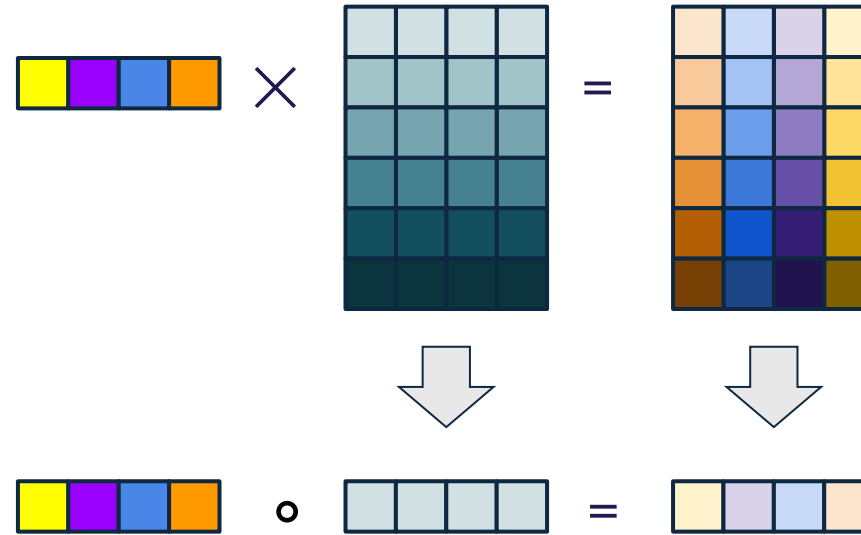
5. Applications and examples

Let's unleash the power of the GPU

Hadamard (element-wise) product

Vector of size 1080

Matrix of size 1080×1920



Who is faster?

- Numpy
- PyTorch
- OpenCL
- 'For loop'



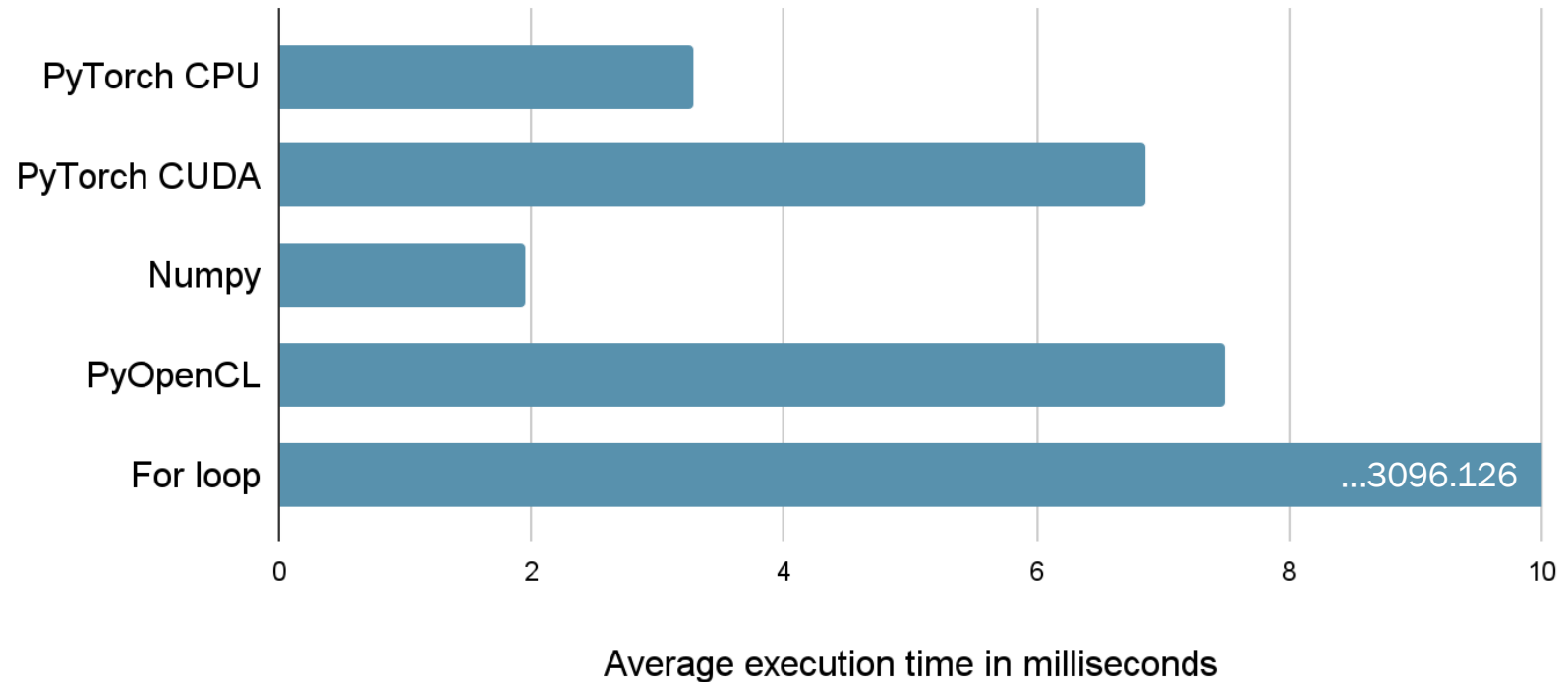
Let's unleash the power of the GPU

Hadamard (element-wise) product

Vector of size 1080

Matrix of size 1080×1920

Performance comparison



Let's unleash the power of the GPU

Other common and simple operations

Average execution time (ms)	PyTorch CPU	PyTorch CUDA	Numpy
Addition	4.205	3.894	1.730
Vector-matrix product	1.184	0.617	4.515
Matrix-matrix product	55.644	325.986	77.976

Matrix operations: signal and image processing

Original

How do I get these images from the original one?

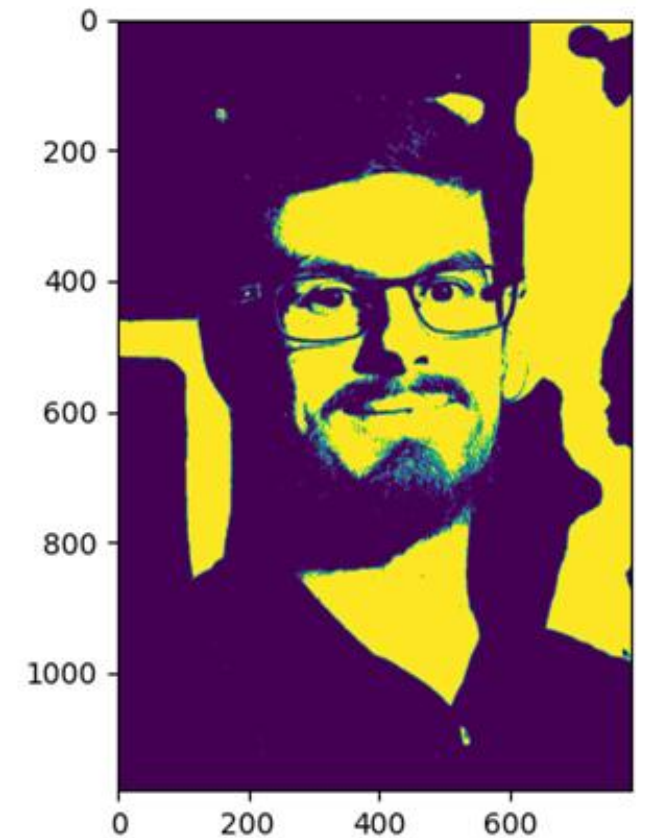
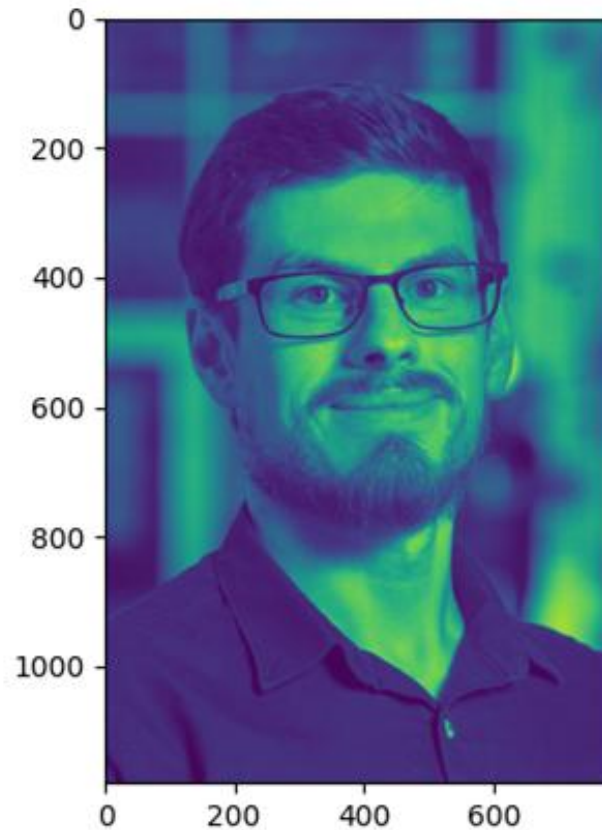
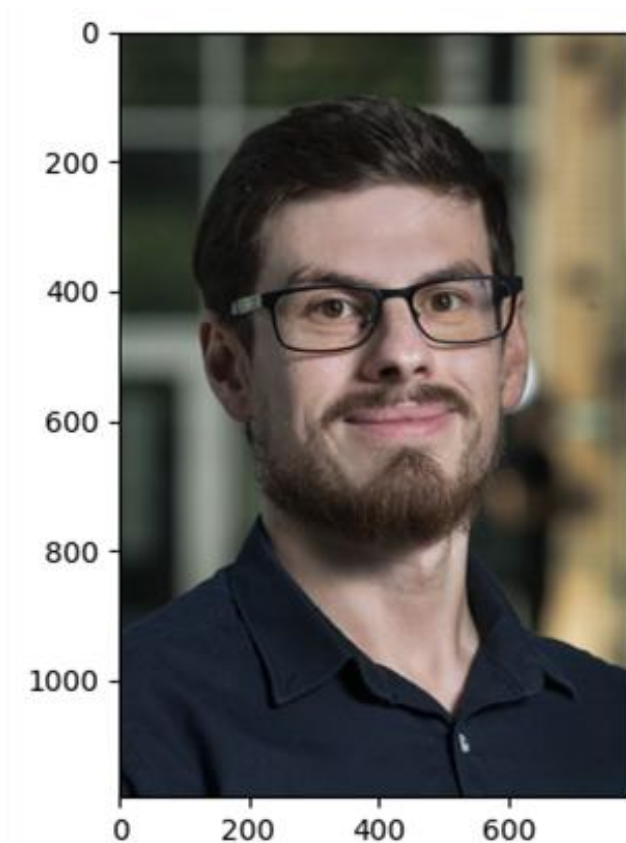


Image manipulation?

Grab a pillow

Pillow allows to import images to numpy arrays in RGB

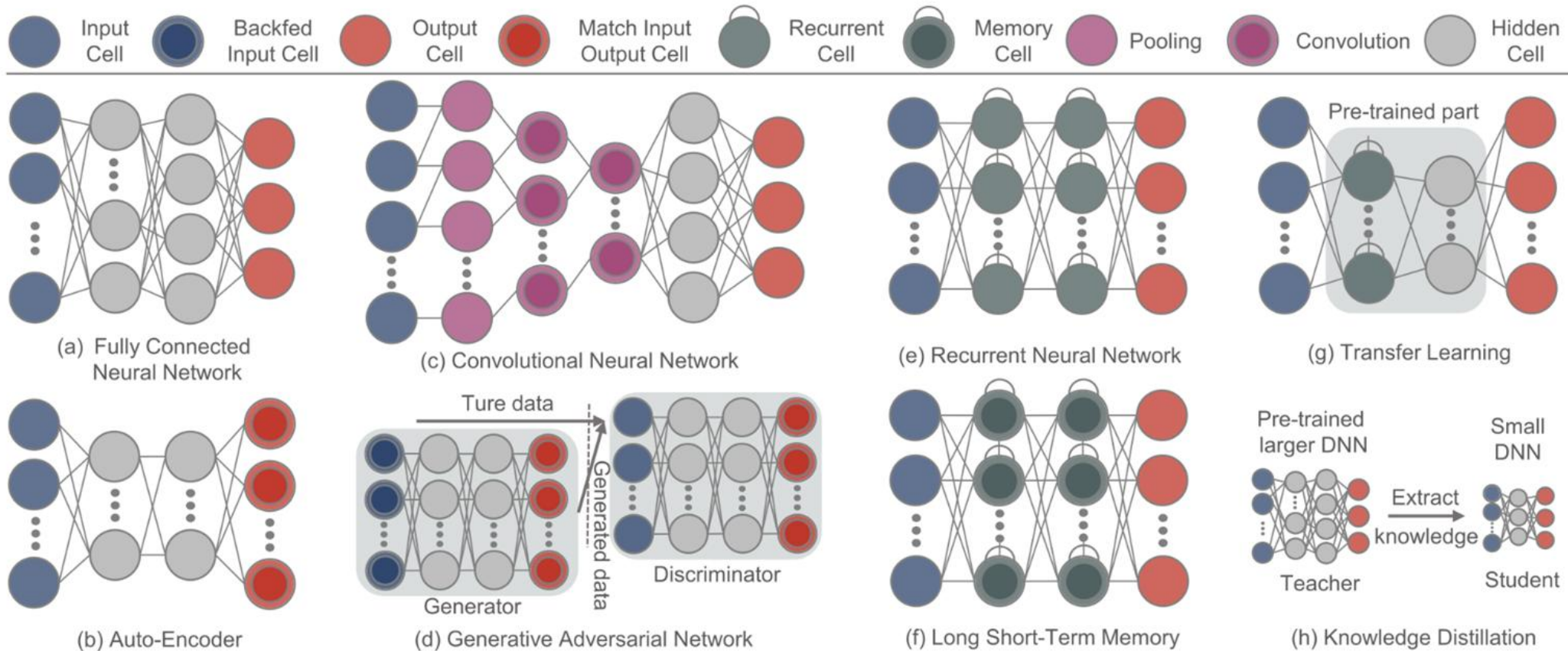
from PIL import Image

```
im = np.array(Image.open('filename.jpg'))
```



pillow

Deep learning: Machine learning with deep neural networks



X. Wang, Y. Han, V. C. M. Leung, D. Niyato, X. Yan, and X. Chen, “**Convergence of Edge Computing and Deep Learning: A Comprehensive Survey,**” IEEE Commun. Surv. Tutorials, vol. 22, no. 2, pp. 869–904, 2020.

Summary and exercises

Summary

GPU programming works best with problems with high levels of parallelism

Simple instructions repeated a massive number of times

Host and device model

Overhead: copying the data to the device and back to CPU

Several ways of using the GPU

NVIDIA CUDA: Greatly simplifies the process but is proprietary

OpenCL: Cross-platform and “hardcore”

Downside: A lot of problems occur during setup

Exercises

1. Vector-matrix multiplication or matrix-matrix multiplication

Implement both algorithms and calculate the speedup with the GPU vs. CPU

In which case the speedup is higher?

2. Image manipulation with GPU

Implement a code to perform a hadamard product (i.e., point-to-point multiplication) between a matrix with random numbers (between 0 and 1) and an image in grayscale.

Use the pillow package to load the image:

```
from PIL import Image
```

```
img = Image.open(file)
```

Does the GPU give you any speedup?

Appendix

Example 1: Initial tests on the kernel

```
prg = cl.Program(context, """
__kernel void only_get_id(
    __global float* output)
{
    int i = get_global_id(0);
    output[i] = i;
}
""").build()
```

What should the output be?

Example 2: Hadamard product

```
__kernel void sum(  
    __global const float *a_g,  
    __global const float *b_g,  
    __global float *result_g)  
{  
    int i = get_global_id(0);  
    result_g[i] = a_g[i] * b_g[i];  
}
```

Let's unleash the power of the GPU

Hadamard (element-wise) product

Who is faster?

- Numpy
- OpenCL
- 'For loop'

Example 3: Inefficient kernel to multiply matrices

```
__kernel void matmul(  
    __global const float *a_g,  
    __global const float *b_g,  
    __global float *result_g)  
{  
    int i = get_global_id(0); # 1st dimension  
    int j = get_global_id(1); # 2nd dimension  
  
    for(int k = 0; k < DIM; k++) {  
        result_g[i*DIM+j] += a_g[i*DIM + k] * b_g[k*DIM+j];  
    }  
}
```

Example 4: A better kernel to multiply matrices

```
__kernel void matmul(  
    __global const float *a_g,  
    __global const float *b_g,  
    __global float *result_g)  
{  
    int i = get_global_id(0); # 1st dimension  
    int j = get_global_id(1); # 2nd dimension  
  
    float result = 0;  
    for(int k = 0; k < DIM; k++) {  
        result += a_g[i*DIM + k] * b_g[k*DIM+j];  
    }  
    result_g[i*DIM+j] += result;  
}
```

That's all for today